Top 75 Frequently asked Angular Interview Questions

1. What is Angular?

Angular is a JavaScript-based framework developed by Google. Angular allows developers to build applications for browsers, mobile, and desktop using web technologies like HTML, CSS, and JavaScript.

Angular2+ is different from Angular1.x and it is completely re-written from scratch using Typescript. At the time of writing this, book the latest version of Angular was 7 which was released on 19 Oct 2018.

2. How to set up the Angular Environment?

Angular is a platform to create Single Page Application (SPA) using Html and Typescript, and main building block is Component, Modules, and Services etc.

To get started with the Angular, we need to set up the development environment, which requires following tools.

- **Nodejs**

To get started with Angular, we need to have node installed; if you don't have nodejs already installed into your machine then you can find the setup.

- **Npm**

Npm stands for Node Package Manager, and it contains the command-line client and using this we can install the various packages into our application.

For that, it manages the package registry with the complete details about the package, name, version, issue etc.

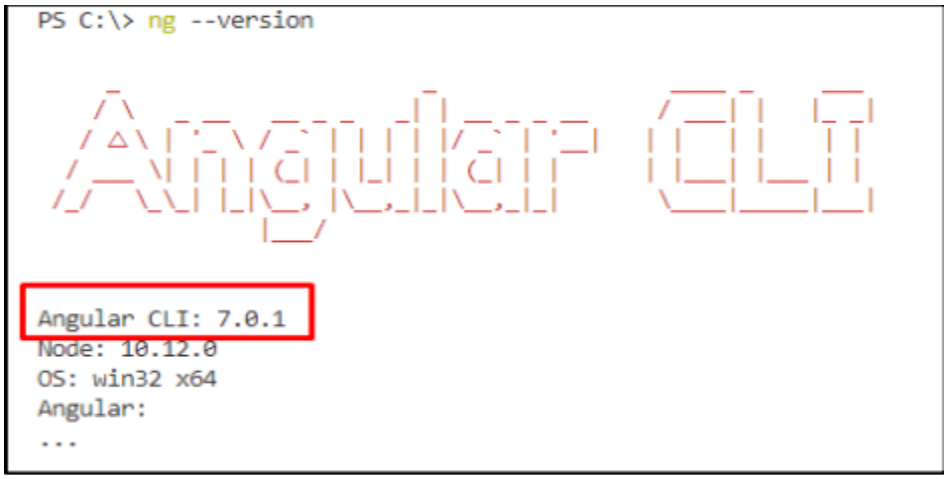
- **Angular CLI**

CLI is one of the important parts while getting started with Angular development and it is used to scaffold and build Angular app faster.It manages the entire tedious tasks like creating files, building the project, serving the project and many more tasks.

To use Angular CLI, we need to install the CLI by using below npm command.

```
npm install -g @angular/cli
```

After installing the CLI, if you want to know the current version then we can find the version like this.



Any text editor tool as IDE

We can choose any of the code editors which are listed above in the IDE question. For Angular development, Visual Studio Code is a nice choice over the other code editors.

After following all of the above steps, now we are ready to get started with the Angular world.

3. What is Angular CLI?

Angular Command Line Interface is a command line tool which is used to create, initialize, scaffold and to manage the whole angular application.

We can also use Angular Console tool to work with the command line to generate or work with different part of the Angular application, using CLI we can manage everything using commands.

You can find the complete details of Angular CLI and different commands into the CLI introduction part later on this series.

4. What are the features of Angular CLI?

Angular CLI is a powerful command-line tool by which we can create new files, update the file, build and deploy the angular application and other features are available so that we can get started with Angular development within few minutes.

- Packaging application and release for the deployment
- Testing Angular app
- Bootstrapping Angular app
- Various code generation options for Component, Module, Class, Pipes, Services, Enums, and many other types are also supported.
- Running the unit test cases
- Build and deployment our Angular application

5. How to create a new Angular app?

To create a new Angular app using CLI command, then we can use below command.

ng new <*your_app_name*>

It will take a few seconds or minutes based on our internet connection speed and then the new angular app will be created with the basic configuration.

6. How to serve the Angular app?

After creating a new Angular app, now it's time to run our app, for that we can use below command to serve our app.

ng serve

After executing above given command, our application will start building and it can also be helpful to rebuild the app after any changes occurred.

If you want to open app automatically into a new window from the browser then we can use the flag –o like this.

ng serve –o

7. How to change default port number other than 4200?

If you have created new Angular app and server the app using ng serve, then our app will be executed on URL localhost:4200, but we can change it using additional flag along with ng serve which is –port.

ng serve --port 4300

8. How to build an Angular application using CLI?

For the deployment purpose, we need to build our code to uglify all JavaScript files and to get ready to submit directory to the server.

Basically, it uses web pack build tools and the regarding data can be fetched from the angular.json file. We can use below command to build the project.

ng build

After executing the above command, the output folder will be created called /dist, but we can change the default directory to store the build output folder.

9. What is Component in Angular?

An Angular component mainly contains a template, class, and metadata. It is used to represent the data visually. A component can be thought as a web page. An Angular component is exported as a custom HTML tag like as:

***<my-app></my-app>*.**

When we create new create an Angular app using CLI, it will create default App component as the entry point of application. To declare a class as a Component, we can use the Decorators and decorator to declare a component looks like the below example.

```
import{Component}from'@angular/core';

@Component({
selector:'my-app',
template: `<h1>{{ message }}</h1>`,
styleUrls:['./app.component.css']
})
exportclassAppComponent{
message = "Hello From Angular";
}
```

For creating a component, we can use decorator @component like the above example. If we are using Angular CLI than new component will be created using the below ng command.

ng generate component componentName

10. What is Data Binding in Angular?

Data binding is one of the important core concepts which is used to do the communication between the component and DOM.

In other words, we can say that the Data binding is a way to add/push elements into HTML Dom or pop the different elements based on the instructions given by the Component.

There are mainly three types of data binding supported in order to achieve the data bindings in Angular.

- One-way Binding (Interpolation, Attribute, Property, Class, Style)
- Event Binding
- Two-way Binding

Using these different binding mechanisms, we can easily communicate the component with the DOM element and perform various operations.

11. What is Interpolation?

Interpolation in Angular is used to show the property value from component to the template or used to execute the expressions.

Generally, interpolation can be used to execute the JavaScript expressions and append the result to the Html DOM.

Another use of Interpolation is to print the data or we can say one-way data, which is coming from the component i.e. from a variable or the item of an array.

And interpolation can be used to achieve Property binding as well by using curly braces {{ }}, let's see a simple example.

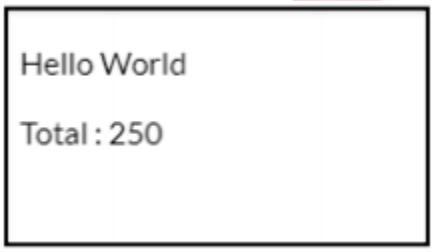
App.component.ts

```
myMessage: string = "Hello World";
```

App.component.html

```
<p>{{ myMessage }}</p>
<p>Total : {{ 25 + 50 + 75 + 100 }}</p>
```

The same way, we can use interpolation to different places or elements like the source of the image, values for the classes, and other use as well. The output will look like this.



12. What is Property Binding?

Property binding is generally used to show or update the value of the variables in component and vice versa. We can also call it one-way data binding thus it is not used to update the two-way binding mechanism, and value will be reflected at the component level.

Let's see the simple example by taking simple string value from component to the template.

App.component.ts

We are going to create a string variable with a default value like this.

myMessage: string = "Hello World";

And to show this message into the template, we will use a {{ }} symbol which is called interpolation like this.

App.component.html

```
<h1>{{ myMessage }}</h1>
```

Whenever we change the value of the myMessage variable, at that time it will be reflected in the template but not vice versa.

13. How to implement Two-way data binding in Angular?

Two-way binding is one of the strongest binding mechanisms, where two different bindings are merged
i.e. input and output bindings.

It means that Event binding works with the two different binding mechanisms at a time and generates the output based on the event which was triggered.

For two-way data binding, we will use the punctuation [( )] in order to bind the two-way data.

It can be happen using the directive called ngModel which listen to the input for any changes and return the output.

We will understand more by implementing the simple example, where we will type the text into the input and at the same time, we will get the value of the same input value.

App.component.html

```
Full Name : <inputtype="text"
[(ngModel)]="fullName"> Your Name is : {{ fullName }}
```

App.component.ts

```
import{ Component } from '@angular/core';

@Component({ selector:'my-app',
templateUrl:'./app.component.html', styleUrls: ['./app.component.css']
})
Export classAppComponent {
fullName: string = '';
}
```

Here in this example, fullName is our variable into the component, and from our template, we are binding the value of textbox and returns to the same variable which holds the current value of fullName.

As we have discussed that it is combination of two binding mechanism which is equivalent to below statement.

```
<input type="text" [ngModel]="fullName" (ngModelChange)="fullName=$event">
```

14. What is Directive in Angular?

The directive in Angular basically a class with the (@) decorator and duty of the directive is to modify the JavaScript classes, in other words, we can say that it provides metadata to the class.

Generally, @Decorator changes the DOM whenever any kind of actions happen and it also appears within the element tag.

The component is also a directive with the template attached to it, and it has template-oriented features. The directive in Angular can be classified as follows.

1. Component Directive
2. Structural Directive
3. Attribute Directive

15. What is Structural Directive in Angular?

The structural directive is used to manipulate the DOM parts by letting us add or modify DOM level elements.

For example, we are adding records every time when we submit the form. To generate a new directive, we can use the following command.

ng generate directive mydirective

And the generated directive file can look like this.

```
// mydirective.directive.ts
import{ Directive } from'@angular/core'; @Directive({
selector:'[appMydirective]'
})
Export class MydirectiveDirective { constructor() { }
}
```

16. What is Attribute Directive in Angular?

Attribute directive is used to change the behavior or the appearance of different elements, and these elements act as an attribute for the DOM elements.

This is the same as custom or other directive but the difference is that we are changing the behavior of elements dynamically.

Let's see the simple directive example and create new directive named attrdemo.directive.ts and source code can look like this.

```
import{ Directive, ElementRef, Input, HostListener } from '@angular/core';
@Directive({
selector: '[appAttdirdemo]'
})
Export classAttdirdemoDirective {
@Input() appAttdirdemo: string; constructor(privateelref: ElementRef) { }
@HostListener('mouseover') onMouseOver() { this.changefontsize(this.appAttdirdemo);
}
@HostListener('mouseleave') onMouseLeave() { this.changefontsize('15px');
}
changefontsize(size) {
this.elref.nativeElement.style.fontSize = this.appAttdirdemo;
}
}
```

In this example, we are going to change the font size of a string using our custom directive, please find below code where I have used directive.

```
<h3 [appAttdirdemo]="'12px'"> This is a Attribute Directive</h3>
```

As you can see that I have used appAttdirdemo which is my directive name and based on a mouse event, it will change the font size.

17. How many types of built-in attribute directives are available?

Basically, there is three type of built-in attribute directives are available to use which are listed below.

- **NgStyle**

To add or remove the different styles of an HTML element

- **NgClass**

To set the different classes of an HTML element

- **NgModel**

We have used NgModel attribute directive which is used to implement two-way data binding.

18. What are various Structural directives in Angular?

Structural directives are used to manipulate the DOM by adding or removing different HTML elements at a time.

We can use the structural directives with the host element, and that it performs the different task based on the directive values.

Probably we can identify the Structural directive, and the main reason is that it contains (*) asterisk as a prefix with all Structural directive.

There are three types of Structural directives are available which are listed below.

- **NgIf**

Can add or remove the element based on the condition

- **NgSwitch**

Similar to switch case and it will render the element based on the matching condition based on the value

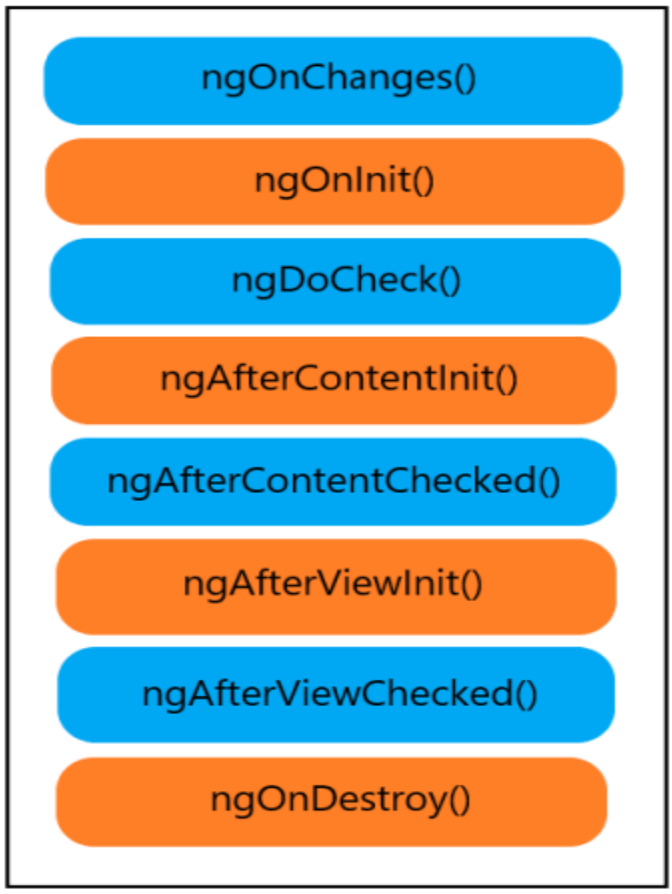
- **NgFor**

Used to repeat the element until it reaches to the total number of items into the array/list

19. What are various Angular Lifecycle hooks?

Angular has its own Component lifecycle, and every lifecycle event will occur based on the data-bound changes.

Right from the initialization of a component to the end of the application, events of the lifecycle are processed based on the current component status.



ngOnChanges()

This is the first method to be called when any input-bound property will be set and always respond before ngOnInit() whenever the value of the input-bound properties are changed.

ngOnInit()

It will initialize the component or a directive and set the value of component's different input properties. And one important point is that this method called once after ngOnChanges ().

ngDoCheck()

This method normally triggered when any change detection happens after two methods ngOnInit() and ngOnChanges().

ngAfterContentInit()

This method will be triggered whenever component's content will be initialized and called once after the first run through of initializing content.

ngAfterContentChecked()

Called after every ngDoCheck() and respond after the content will be projected into our component.

ngAfterViewInit()

Called when view and child views are initialized and called once after view initialization.

ngAfterViewChecked()

Called after all of the content is initialized, it does not matter either there are changes or not but still this method will be invoked.

ngOnDestroy()

Called just before destroys of a component or directive and we need to unsubscribe any pending subscription which can harm us as extreme memory leaks.

20. Methods to Share Data between Angular Components

While developing an application, there is a possibility to share data between the components.

And sharing data between the components is our day-to-day kind of stuff where communication between components is a must either parent to child, child to the parent or communicate between un-related components.

To enable data sharing between components, different ways are possible which are listed below.

- Sharing data using @Input()
- Sharing data using @Output and EventEmitter
- Sharing data using @ViewChild
- Using Services

21. How to share data using @ViewChild ()

This is the way to share the data from child to parent component using @ViewChild () decorator. ViewChild is used to inject the one component into another component using @ViewChild() decorator.
One thing to keep in mind about the ViewChild is that we need to implement AfterViewInit lifecycle hook to get the data because the child won't be available after the view has been initialized.

Let's see one simple example that how to use @ViewChild to inject the child component into the parent
Component.

```
// child.component.ts
import { Component } from '@angular/core';

@Component({
```

```
selector: 'app-child',
template: `<h3>{{ counter }}</h3>`
})
export classChildComponent { counter: number = 0;
increment() {
this.counter++;
}
decrement() {
this.counter--;
}
}
```

Here in this example, we are going to implement the counter example, and into the child component, there are two different methods to increment and decrement the counter value.

Now let's inject the child component into the parent using ViewChild like this.

```
// app.component.ts
import { Component, ViewChild } from '@angular/core'; import { ChildComponent } from
'./child/child.component';

@Component({
selector:'my-app',
templateUrl:'./app.component.html', styleUrls: ['./app.component.css']
})
export classAppComponent { @ViewChild(ChildComponent)
privatechildComponent: ChildComponent;

add() {
this.childComponent.increment();
}
sub() {
this.childComponent.decrement();
}
}
```

In our app component, we have injected child component using ViewChild and using different methods of child increment () and decrement (), we can add or remove the counter value directly into our parent component.

```
<!-- app.component.html -->
<button type="button" (click)="add()">+</button>
<app-child></app-child>
<button type="button" (click)="sub()">-</button>
```

When we try to click on the plus or minus button, the counter value will be reflected, this is how we can inject the child component functionality into the parent component using @ViewChild ().

22. What is Module in Angular?

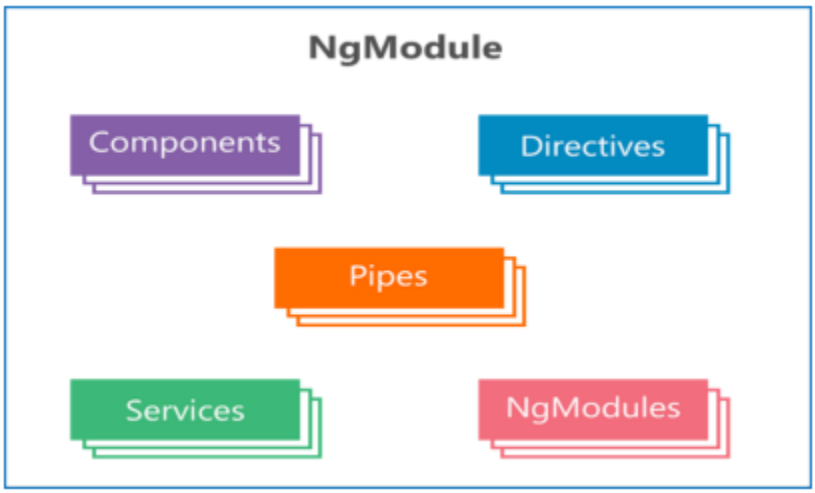
The module in Angular is a very essential part of the application, which is used to organize our angular app into a distributive manner.

Or we can say that Modules plays an important role to structure our Angular application.

Each Angular application must have one parent module which is called app module and it contains the list of different items which are listed below.

- Components
- Service providers
- Custom modules
- Routing modules
- Pipes

Module file group all of them into a single file; the same module will be useful for bootstrapping the application.



23. How to create a new Module using Angular CLI?

To get started with the modules, we need to create the module file by using below ng command.
ng generate module mycustommodule

24. What different types of Modules are supported by Angular?

Modules in Angular are categorized into 5 types which are listed below.

- Routing module

- Service module
- Features module
- Widget module
- Shared module

25. How to create a Module with routing using CLI?

We can configure the routes along with the Modules by using the command, which generates the complete routing file as explained below.

ng generate module mymodule--routing

Here in this command, you can notice that we have used the additional flag –routing which is used to create a Module for the routing as well as the normal module file.

After running above command, it will create one folder along with the two files.

1. Mymodule-routing.module.ts

```
import{ NgModule } from '@angular/core';
import{ Routes, RouterModule } from '@angular/router'; const routes: Routes = [];
@NgModule({
imports: [RouterModule.forChild(routes)], exports: [RouterModule]
})
Export classMymoduleRoutingModule{ }
```

2. Mymodule.module.ts

```
import{ NgModule } from'@angular/core'; import{ CommonModule } from'@angular/common';
import{ MymoduleRoutingModule } from'./mymodule-routing.module';

@NgModule({ declarations: [],
imports: [
CommonModule, MymoduleRoutingModule
]
})
Export classMymoduleModule{ }
```

26. What is Entry Component?

The bootstrapped component is an entry component which loads the DOM during the bootstrapping process. In other words, we can say that entry components are the one, which we are not referencing by type, thus it will be included during the bootstrapping mechanism.

There are mainly two types of entry component declarations can be possible.

1. Using bootstrapped root component

```
import{ NgModule } from'@angular/core'; import{ CommonModule } from'@angular/common';
import{ MymoduleRoutingModule } from'./mymodule-routing.module';

@NgModule({ declarations: [], imports: [
CommonModule, MymoduleRoutingModule
]
})
Export classMymoduleModule{ }
```

We can specify our component into bootstrap metadata in an app.module file like this.

2. Using route definition

```
Const myRoutes: Routes = [
{
path:'',
component: DashboardComponent
}
];
```

Another way to define the entry component is to use it along with the routing configuration like this.

27. What is Pipe in Angular?

During the web application development, we need to work with data by getting it from the database or another platform, at that we get the data into a specific format and we need to transform it to a suitable format, and for this purpose, we can use pipes in Angular.

The pipe is a decorator and it is used to transform the value within the template.

It is just a simple class with @Pipe decorator with the class definition and it should implement the PipeTransform interface which accepts the input value and it will return the transformed value.
To create a new pipe, we can use the below command.

28. What is the use of PipeTransform Interface?

When we want to create custom pipe at a time we should implement an interface named PipeTransform which accepts input and returns the transformed value using transform() method.

There are two parameters provided with the transform () method.

· **Value -** A value to be transformed

· **Exponent -** Additional arguments along with the value like size, volume, weight etc.

29. What are built-In Pipes in Angular?

There are different types of in-built pipes available in Angular, which are listed below.

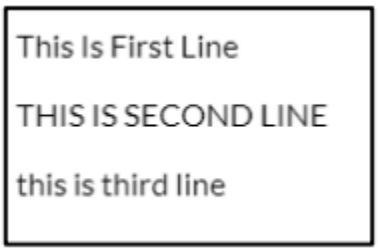
- Date pipes
- Currency pipes
- Uppercase pipes
- Lowercase pipes
- Titlecase pipes
- Async pipes
- Slice pipes
- Number pipes
- Json pipes

And many more types of pipe are available to use for data transformation.

30. What is a Pure Pipe?

By default, in Angular, pipes are pure, and every pipe created in the angular are pure by nature.

The pure pipe will be used only when it requires pure changes in terms of the value, and value can be String, Number, and Symbol etc. And other object values like Array, Functions, and others as well.



31. Explain Impure Pipe using an example?

We know that by default every pipe we create is a pure pipe, but we can also create impure pipe by providing pure to false as describe below.

Impurepipe.pipe.ts

```
import{ Pipe, PipeTransform } from '@angular/core';

@Pipe({ name:'impurepipe', pure:false
})
Export classImpurepipeimplementsPipeTransform {
transform(value: any, args?: any): any {
if(!value) {
return value = 'Value Not Found';
```

```
}
return value = value;
}
}
```

In this example, you can see that using pure option we can set it to true or false, so if we want to set a pipe to impure than value would be false just like this example.

32. What is Routing in Angular?

Routing in Angular allows the user to navigate to the different page from the current page. When we try to enter URL in the browser at a time it navigates to a specific page and performs appropriate functions implemented into the pages.

Routing is one of the crucial tasks for any application, because the application may have a large number of pages to work with.

33. What is RouterModule?

RouterModule in Angular provides different directives and providers to navigate from one page to another page.

And this module makes sure that navigation transforms the URL in the correct manner and also manages different URL transitions.

Soon, we will cover the actual use of RouterModule later on this series of Routing.

34. How to use Routing in Angular?

Angular has its own library package @angular/router it means that routing is not a part of core framework @angular/core package.

In order to use Routing in our Angular application, we need to import RouterModule like this into our root module file.

```
import{ Routes, RouterModule } from'@angular/router';
```

35. What is the Route in Angular?

The route is the Array of different route configuration and contains the various properties which are listed below.

Property	Description
Path	String value which represents route, matcher
pathMatch	Matches the path against the path strategy
Component	The component name which is used to indicate the component type
Matcher	To configure custom path matching
Children	Children are the array of different child routes
loadChildren	Load children are used to loading the child routes as lazily loaded routes

These are the primary properties and also other properties are supported like data, canActivate, resolve, canLoad, canActivateChild, canDeactivates.

And by using these different properties, we can perform many operations like loading the child route, matching the different route paths; specify the route path for specific component etc.

36. What are different types of methods in RouterModule?

There are mainly two static methods are available which are listed below.

1. **forRoot():** It performs initial navigation and contains the list of configured router providers and directives. You can find the simple example using forRoot below.

```
RouterModule.forRoot([
{ path:'', redirectTo:'dashboard', pathMatch:'full' },

{ path:'dashboard', component:DashboardComponent },
{ path:'about', component:AboutComponent },
```

```
{ path:'contact', component:ContactComponent }
])
```

2. **forChild():** forChild is used to create the module with all of the router directive and provider registered routes.

37. What isRouterLink?

RouterLink in Angular is a Directive and used to provide a link to the specific route.

We use routerLink to transfer the route from one component to another by identifying the route value which is configured in the router module.

The basic syntax of routerLink will be like this.

```
<a [routerLink]="['/home']"> Home Page</a>
```

38. What is RouterOutlet?

RouterOutlet is a directive provided by Angular which is used to load the different components based on the router state.

Whenever the user clicks on the link, at a time the activated component will be rendered and added to HTML DOM inside the router-outlet directive.

To load the different components, we need to use a directive like below into our HTML template.

```
<router-outlet></router-outlet>
```

Here it is the simple example, to load different components inside the router-outlet.

```
<div class="container">
<a routerLinkActive="active" routerLink="/page1">Page 1</a> |
<a routerLinkActive="active" routerLink="/page2">Page 2</a> |
<a routerLinkActive="active" routerLink="/page3">Page 3</a>
<router-outlet></router-outlet>
</div>
```

39. Explain the types of Route loading

In Angular, basically, there are three types can be used for Route loading which is listed below.

- Pre-Loading
- Eager Loading
- Lazy Loading

40. What is Pre-Loading?

Preloading is a completely different thing than lazy loading and eager loading, preloading means to load the module after any eager module will be loaded, and after eager module loaded at startup, the router looks for the modules which are unloaded and it can be preloaded.

Let's understand by example, assume that we have three different module Home, About and Contact.

Now Home is the first page when our app will be executed, so remaining two modules will remain unloaded, but at some point of time user may want to contact about the business owner to contact them using contact page so that contact page should be ready to load after the Home page.

We can say that the Homepage will be loaded first, then after contact page should be preloaded afterward, which is called pre-loading.

41. What is Eager Loading?

We have seen an example that how preloading works, but there is the difference between both of them is Eagerly loaded module will be executed at a time of application startup.

By default, all the modules in our application are eagerly loaded it means every module will be loaded in the beginning like we have three modules Home, about and Contact than every module will be loaded which is completely opposite to the lazy loading approach.

For example, we have three components which are eager loaded.

// app-routing.module.ts

```
import{ NgModule } from '@angular/core';
import{ Routes, RouterModule } from '@angular/router'; import{ HomeComponent } from
'./home/home.component'; import{ AboutComponent } from './about/about.component';
import{ ContactComponent } from './contact/contact.component';

const routes: Routes = [
{
path:'', redirectTo:'home', pathMatch:'full',
},
{
path:'home', component:HomeComponent
},
{
path:'about', component:AboutComponent
},
{
path:'contact', component:ContactComponent
},
];
```

```
@NgModule({
imports: [RouterModule.forRoot(routes)], exports: [RouterModule]
})
Export classAppRoutingModule{ }
```

We have three different components, Home, About and Contact, so whenever application loaded at that time all the modules will be loaded at application startup.

42. What is Lazy Loading?

We are loading every module at once, but do you know that the performance of the application may decrease, for that we need to load only necessary modules which required for application startup.

When the user navigates to a specific route and the component and module are loaded for the single page is called lazily loaded modules and it will be directly reflected the app bundle size.

Here in this question, we are talking about Lazy loading, in which the current screen will only be loaded and rest of the screen will not be loaded at application startup, we can also call it On-Demand loading.

Let's see one simple example of a routing module where we are loading the lazily loaded module, for that let's assume we have three different pages Home, About and Contact pages.

```
import{ NgModule } from '@angular/core';
import{ Routes, RouterModule } from '@angular/router'; import{ HomeComponent } from
'./home/home.component';
const routes: Routes = [
{
path:'', redirectTo:'home', pathMatch:'full',
},
{
path:'home',
component: HomeComponent
},
{
path:'about', loadChildren:'./about/index.module#AboutModule'
},
{
path:'contact', loadChildren:'./contact/index.module#ContactModule'
},
];
@NgModule({
imports: [RouterModule.forRoot(routes)], exports: [RouterModule]
})
Export classAppRoutingModule{ }
```

Consider the above example where we have three different components, but we have one component
HomeComponent which is eagerly loaded when we start our app.

And rest of the components are lazy loaded and their respective module file will be loaded whenever users click on their links, it will affect the size of the bundle as well as a performance by reducing initial loading modules.

43. What is Angular Form?

During developing the applications, we need to handle the different inputs and files like handling user login, registration; getting data for a different purpose and other forms are as well.

At that time Angular forms are used to handle the user inputs. For that there are two different approaches are provided which are listed below.

1. Reactive Forms
2. Template-driven Forms

By using these two approaches, we can create forms, implement various input validations, and track the input changes.

44. What are the Building blocks of an Angular Form?

While working with the Forms in Angular, we are using different form controls to get the user inputs, based on that we have some basic building blocks which are listed below.

- **FormGroup**

It has the collection of all FormControls used specifically for a single form.

- **FormControl**

FormControlis used to manage the form controls value and also maintain the status of the validation.

- **FormArray**

Manages the status and value for Array of the form controls

- **ControlValueAccessor**

Acts as a mediator between FormControl and our different native DOM elements.

45. What is Reactive Form in Angular?

One of the widely used forms approach which is called Reactive forms, in which structure of the form will be implemented in the code I mean to say inside the component itself.

Reactive forms approach is robust, scalable, and one of the best parts is that it is reusable and it is also called Model-driven forms.

In order to use Reactive forms, we need to import module as described below.

```
Import { ReactiveFormsModule } from '@angular/forms';
//And then we need to add this module into import array like this.
imports: [ BrowserModule, CommonModule, ReactiveFormsModule
],
```

46. What is Template-driven Form in Angular?

This is the simplest approach to use forms in Angular, and it will be useful if you have simple forms which can be managed within your template but it cannot be scalable as compared to Reactive forms.

In Template-driven forms, Angular will create the models i.e. FormGroups and FormsControl sand then it uses ngModel to enable template-driven forms approach.

To use template-driven approach, we need to import forms module like this into module file.

```
import{ FormsModule } from '@angular/forms';
//And also need to add inside imports array like this.
imports: [ BrowserModule, CommonModule, FormsModule
],
```


47. What are the differences between Reactive form and Template-driven form in Angular?

The differences between Reactive Form and Template-driven Form are given below:

Reactive Forms	Template-driven Forms
Supports dynamic form and structure	Supports static form approach
Form structure will be implemented in Typescript or else you can say in the code itself	Form structure will be implemented in the HTML template
Form values passed to the code using forms value property	It allows one-way and two-way data binding to pass the forms value
Behavior is Synchronous	Behavior is Asynchronous

48. What is FormControl in Angular and how to use it?

To create a reactive form, we use FormGroup, FormControl, and FormArray with the ReactiveFormsModule.

FormControl is generally used to track the form values and also helps to validate the different input elements. To use FormControl, we need to import it into the module file like this.

```
Import { FormsModule, ReactiveFormsModule } from'@angular/forms';
```

Also, we need to add the same inside the imports array.

```
imports: [ BrowserModule, FormsModule, ReactiveFormsModule],
```

To use FormControl with the template, you can see the simple example like this.

```
<table>
<tr>
<td>Enter Your Name :</td>
<td><input type="text" [formControl]="fullName"/></td>
</tr>
</table>
```

And inside the component file, we need to import FormControl and create new instance as described below.

```
import{ Component } from '@angular/core';
import{ FormControl, Validators } from '@angular/forms';
@Component({ selector:'my-app',
templateUrl:'./app.component.html', styleUrls: [ './app.component.css' ]
})
Export classAppComponent  {
fullName = new FormControl('', Validators.required);
}
```

49. What is FormGroup and how to use it in Angular?

FormGroup in Angular is a collection of different FormControls and it is used to manage the value of different inputs and implement validations as well.

In other words, we can say that FormGroup is a group of different elements which are the instance of FormControl.

To use FormGroup, we need to import it into the component like this.

import{ FormGroup, FormControl, Validators } from '@angular/forms';

And to use FormGroup with the different FormControls, here it is the simple structure which you can follow.

```
demoForm = new FormGroup({
firstName :new FormControl('Manav', Validators.maxLength(10)), lastName:new
FormControl('Pandya', Validators.maxLength(10)),
});
```

Here in this example, we have two different form controls firstName and lastName which in the textbox to get the values from the end user.

Now we will use formControlName which is used to sync the input value with the FormControl and vice versa.

```
<form [formGroup]="demoForm" (ngSubmit)="submit()">
<table>
<tr>
<td>First Name :</td>
<td><input type="text" formControlName="firstName"/></td>
</tr>
<tr>
<td>Last Name :</td>
<td><input type="text" formControlName="lastName"/></td>
</tr>
<tr>
<td><input type="submit"/></td>
</tr>
</table>
</form>
```

After running the above example, we will get the output like this.

50. What are various built-in validators in Angular?

Angular provides a set of inbuilt validators which we can use directly based on the situations and these validators are.

- required
- min
- max
- requiredTrue
- minLength
- maxLength
- email
- compose
- composeAsync
- nullValidator

51. How to create custom validation in Angular?

Sometimes it may possible that these inbuilt validators would not be helpful to accomplish our validation requirements, so for that, we can also create our custom validations.

Custom validation is as it's nothing more than regular function, let's see one simple example in which salary should be more than 10000 and for that implement function like this.

```
functionisSufficientSalary (input: FormControl) {
const isSufficientSalary = input.value>10000;
return isSufficientSalary ?null : { isSufficient :true };
}
```

Here in this example, we have implemented an isSufficientSalary function which takes an input value and if the value is not more than expected value than it returns false, and to use this function inside component like this.

```
this.demoForm  = this.builder.group({
salary:new FormControl(", [ Validators.required, isSufficientSalary ]),
});
```

As you can see that we have used two different validators, first is required validator and another is our custom validator, find the template code below.

```
<form [formGroup]="demoForm" (ngSubmit)="submit()">
<table>
<tr>
<td>
<label for="salary">Salary</label>
</td>
<td>
<input type="text" name="salary" id="salary" [formControlName]="'salary'"><br/>
```

```
</td>
<td>
<div [hidden]="!demoForm.get('salary').hasError('isSufficient')"> Salary is not sufficient
</div>
</td>
</tr>
<tr>
<td><input [disabled]="!demoForm.valid" type="submit"></td>
</tr>
</table>
</form>
```

And when we run this demo, you can see the output like this.

I am going to use salary 10000 which is not valid and see the error message.

When I'm going to provide the expected value then it will be accepted.

52. What is Dependency Injection in Angular?

You may have heard this term called Dependency Injection so many times, but many of them don't know the actual importance and meaning about that.

Dependency Injection is an application design pattern which is used to achieve efficiency and modularity by creating objects which depend on another different object.

In AngujarJs 1.x, we were using string tokens to get different dependencies, but in the Angular newer version, we can use type definition to assign providers like below example.

```
constructor(private service: MyDataService) { }
```

Here in this constructor, we are going to inject MyDataService, so a whenever instance of the component will be created at that time it determines that which service and other dependencies needed for a component by just looking the parameters of the constructor.

In short, it shows that current component completely depends on MyDataService, and after instance of the component will be created than requested service will be resolved and returned, and at last constructor with the service argument will be called.

53. What are many ways to implement Dependency Injection in Angular?

In order to implement DI, we should have one provider of service which we are going to use, it can be anything like register the provider into modules or component.

Below are the ways to implement DI in Angular.

- Using @Injectable() in the service
- Using @NgModules() in the module
- Using Providers in the component

54. What are Observables in Angular?

Observables is a way to populate the data from the different external resources asynchronously.

Observable replaces the promises in most of the cases like Http, and the major difference between the Promises and Observable is that Observable can be used to observe the stream is different events.

Observable is declarative it means that when we define a function for publishing but it won't be executed until any consumer subscribes it.

Due to asynchronous behavior of Observable, it is used extensively along with Angular because it is faster than Promise and also can be canceled at any time.

55. What is RxJS?

RxJS stands for Reactive Extensions for JavaScript, and it is a popular library which is used to work with the asynchronous data streams.

While working large and complex application, RxJS can help developers to represent multiple asynchronous stream data and can subscribe to the event streams using the Observer.

When an event occurs, at that time Observable notifies the subscribed observer instance.

In order to use RxJS into Angular application, there is an npm package and by installing package we can import a library like this.

```
import { interval } from 'rxjs';
```

Interval is just a function into RxJS library, but it has a different set of function which we can use into our application.

56. What is a Subscription?

Subscription is a kind of disposable resources such as execution of the Observables.

While working with the Observables, the subscription has one method called unsubscribe () which is used to release the different resources held by the subscription and it won't take arguments.

For example, we care calling the GET api for getting the employee details using the services.

```
import { Component } from'@angular/core'; import { Subscription } from'rxjs';
import { MyDataService } from'./mydataservice.service';

@Component({
selector: 'my-app',
templateUrl: './app.component.html', styleUrls: ['./app.component.css']
})
exportclass AppComponent {

employee: string; subscription: Subscription;
constructor(private service: MyDataService) {
 }
ngOnInit() {
this.subscription = this.service.getEmployees()
.subscribe((employee: string) =>this.employee = employee);
}

ngOnDestroy(): void { this.subscription.unsubscribe();
}
}
```


In this example, we are getting the data using service method and subscribed it. But it is custom observable so that we can manually unsubscribe the Observable when our component gets destroyed.

The best place to unsubscribe the subscription is ngOnDestroy lifecycle hooks because we need to destroy the resources which are held by the subscribers.

57. What is Angular Service?

While developing app we need to change or manipulate data constantly, and in order to work with data, we need to use service, which is used to call the API via HTTP protocol.

Angular Service is a simple function, which allows us to create properties and method to organize the code.

To generate services, we can use below command which generates service file along with default service function structure.

ng generate service mydataservice

58. Why we should use Angular Services?

Services in Angular are the primary concern for getting or manipulate data from the server. The primary use of the services is to invoke the function from the different files like Component, Directives and so on.

Normally services can be placed separately in order to achieve reusability as well as it will be easy to distribute the Angular application into small chunks. Using dependency injection, we can inject the service into the Component's constructor and can use different functions of service into the whole component.

One of the main advantages is Code Reusability because when we write any function inside the service file then it can be used by the different component so that we don't need to write same methods into multiple files.

59. What is Singleton Service?

Singleton Service in Angular means to provide Service reference to application root, it can be created using two different ways.

· Declared service should be provided in the application root

```
import{ Injectable } from'@angular/core';

@Injectable({
providedIn:'root'
})
```

```
Export classMydataserviceService { constructor() { }
}
```

Add inside AppModule or another module which indirectly imports into the AppModule

```
import{ NgModule } from '@angular/core'; import{ CommonModule } from
'@angular/common'; import{ AppComponent } from './app.component';
import{ Mydataservice } from './mydataservice.service';

@NgModule({ declarations: [
AppComponent
],
imports: [
CommonModule
],
```

```
exports: [], providers: [
ApiService, // Singleton Services
]
})

ExportclassAppModule{ }
```

60. What is HTTP Client in Angular?

While working with any web-application, we always need the data to work with and for that, we need to communicate with the database which resides on a server.

In order to communicate from the browser, it supports two types of API.

- Fetch() API
- XMLHttpRequest

Same way in Angular, we have HTTP API which is based on XMLHttpRequest which is called HttpClient to make a request and response.

Before Angular 5, HTTP Module was used but after that, it was replaced by the HttpClientModule. To use HttpClientModule in our Angular app, we need to import it like this.

```
import{ HttpClientModule } from '@angular/common/http';
```

61. How many types of HTTP Requests are supported?

There are total 8 types of methods are supported which are listed below.

- GET Request
- Post Request
- Put Request
- Patch Request
- Delete Request
- Head Request
- Jsonp Request
- Options Request

62. How to inject HttpClient in our Angular Service?

After importing theHttpClientModule into the root module file, now we will be able to inject HttpClient into our service which looks like this.

```
import{ Injectable } from'@angular/core';
// Importing HttpClient
```

```
import{ HttpClient } from'@angular/common/http';

@Injectable()
Export classConfigService {
// Injecting HttpClient constructor(private http: HttpClient) { }
}
```

Now using HTTP, we will be able to use different Http request methods like Get, Post, Put, Delete and other methods as well into our whole service file.

63. What is Interceptor?

The inception of HTTP request and response is one of the major parts of HTTP package and using inceptor we can transform the HTTP request from our angular application to the communication server, and the same way we can also transform the response coming from the server.

If we don't use the interceptors then we need to implement the tasks logic explicitly whenever we send the request to the server.

64. How to create Interceptor in Angular?

In order to use Interceptor, we need to implements HttpInterceptor interface, and then each request needs to be intercepted using intercept () method as described below.

```
import{ Injectable } from '@angular/core';
// Importing interceptor
import{ HttpEvent, HttpInterceptor, HttpHandler, HttpRequest } from @angular/common/http';
import{ Observable } from 'rxjs';

@Injectable()
Export classNoopInterceptorimplementsHttpInterceptor {

// Intercept method
intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
return next.handle(req);
}
}
```

As you can see that we have implements the interface HttpInterceptor which is an essential part of implementing interceptor, you may be confused about HttpInterceptor but soon in this series, we are going to cover these topics with an example.

65. What is the use of HttpHeader in Angular?

While sending a request to the server, sometimes we need to pass some extra header for different uses like content type specific to JSON, authorization etc.

For that we can use HttpHeaders to provide header data with the requests, please find below example to understand its simple use.

Import the HttpHeaders like this.

Import { HttpHeaders } from '@angular/common/http';

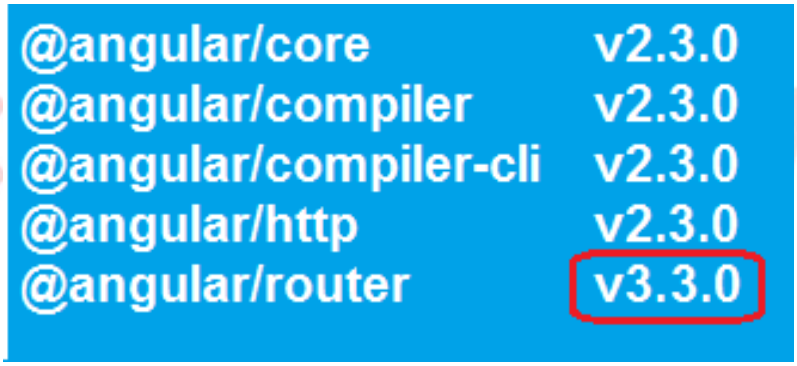
To provide the different header data, we can use HttpHeaders like this.

```
headers: newHttpHeaders({
'Content-Type':'application/json', 'Authorization':'authtoken'
})
```

Here in this example, we have used content-type as JSON and also, we are passing authorization header along
with our request.

66. Why was version 3 skipped? 9

To avoid the confusion due to the miss alignment of the router's package version which is already distributed as version 3.3.0 with Angular 2 contain router file version 3.3.0



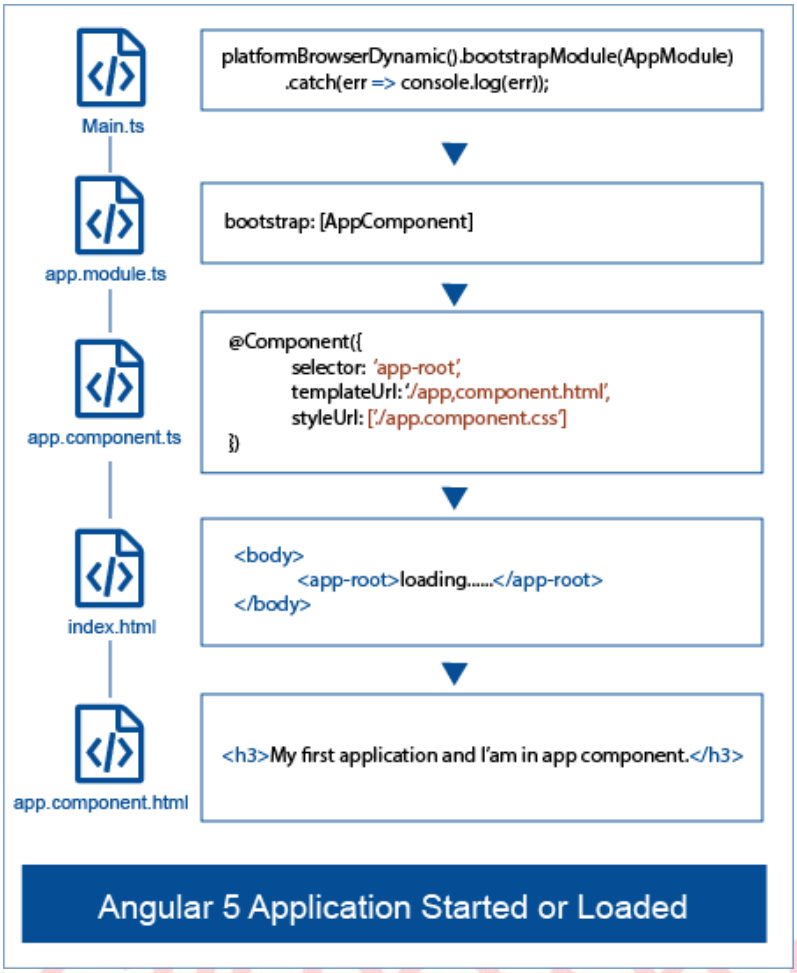
67. How can we run two Angular project simultaneously?

For running an Angular project, we use the following command.
ng serve

With the help of this command, the Angular project runs on port number 4200, i.e., localhost:\\4200. Now, if we want to run another project or want to change the port number of the same project, then we can use the following command.
ng serve --port 4210

68. How does an Angular application get start?

An Angular application gets loaded or started by the below ways.



69. What is webpack?

Webpack is a tool used by an Angular CLI to create bundles for JavaScript and stylesheets. Webpack injects these bundles into index.html file at runtime.

70. What is the difference between AOT and JIT?

In Angular we have AOT (Ahead of Time) and JIT (Just In Time) Compilation for compile the application. There are few differences between JIT and AOT:

- JIT stands for Just In Time so by the name you can easily understand its compile the while AOT stands for Ahead of Time so according to the name
- According to compilation, JIT loads the application slowly while application more quickly as compared to that of JIT.
- In case of JIT, Template Binding errors will show at the time of showing the application while in case of AOT Template binding errors will show at the building time.
- In case of because in case of AOT the bundle

71. What are the commands for the following?

For creating new component	ng g c MyComponentName
For creating new service	ng g s MyServiceName
For creating new module	ng g m MyModuleName
For creating new directive	ng g d MyDirectiveName
For creating new pipe	ng g p MyPipeName
For creating routing guard	ng g g GuardName
For creating class	ng g cl MyClassName
For creating interface	ng g i MyInterfaceName
For creating enum	ng g e MyEnumName

72. What is the purpose of main.ts file?

The main.ts file is the main file which is the start point of our application. As you have read before about the main method the same concepts are here in the Angular application. It's the file for the bootstrapping the application by the main module as

.bootstrapModule (AppModule). It means according to main.ts file, Angular loads this module first.

73. What is ViewEncapsulation?

ViewEncapsulation decides whether the styles defined in a component can affect the entire application or not. There are three ways to do this in Angular:

Emulated: When we will set the ViewEncapsulation.Emulated with the @Component decorator, Angular will not create a Shadow DOM for the component and style will be scoped to the component.One thing need to know necessary it's the default value for encapsulation.

Native: When we will set the ViewEncapsulation.Native with the @Component decorator,Angular will create Shadow DOM for the component and style will create Shadow DOM from the component so style will be show also for child component.

None: When we will set the ViewEncapsulation.None with the @Component decorator, the style will show to the DOM's head section and is not scoped to the component. There is no Shadow DOM for the component and the component style can affect all nodes in the DOM.

74. What is nested component?

When we will create any two component as first component and second component. Now if we will use one component selector within another component so its called nested component

For example:If we have one component as:


```
import { Component } from '@angular/core';

@Component({
 selector: 'app-second',
 template: `
 <h2>My second Component</h2>
 `
})
export class SecondComponent {
 constructor() {}
}
```

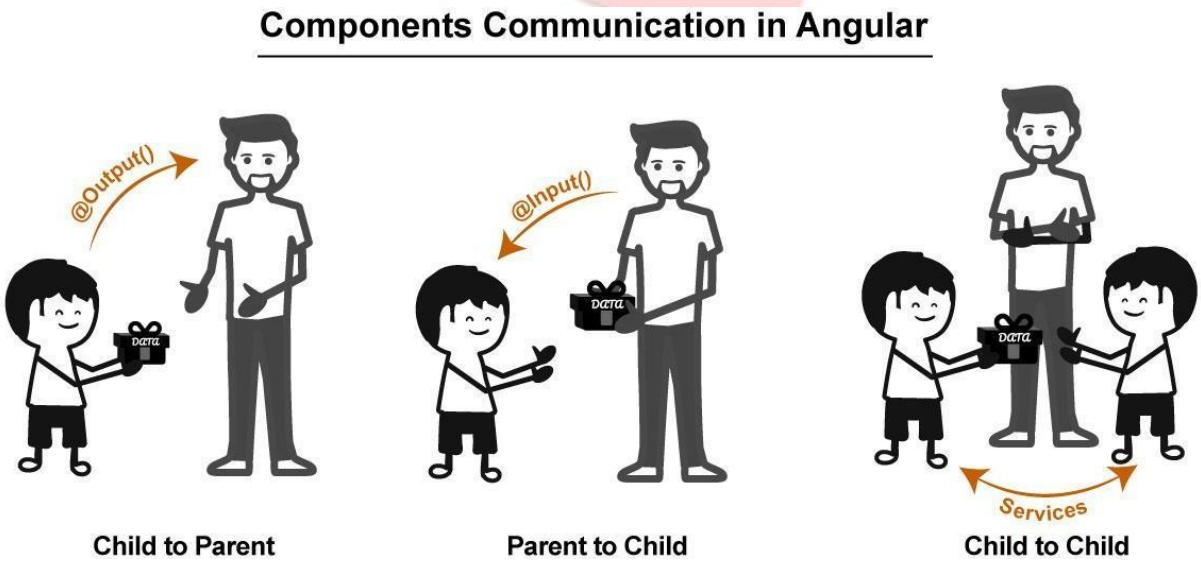
Another Component as:

```
import { Component } from '@angular/core';

@Component({
 selector: 'app-first',
 template: `
 <h2>My First Component</h2>
 <app-second></app-second>
 `
})
export class SecondComponent {
 constructor() {}
}
```

You can see in above, I am using the selector of one component with in another component so it's the example of nested component. We can say one component as Parent as in above First component where we have added the selector of second component and child as Second component.

75. How can we pass data from component to component?



We can pass the data from component either parent and child relationships or not. If component are separate and we want to pass data between component so we can use services with get and set properties.set for setting the values which you want and get for getting the values which added by one component .